2D3PO: A DISTRIBUTED COMPUTING REINFORCEMENT LEARNING FRAMEWORK FOR PROXIMAL POLICY OPTIMIZATION

***Abstract*—This paper presents 2D3PO, a modular distributed reinforcement learning (RL) framework and toolkit for Proximal Policy Optimization (PPO) that integrates Ray-based environment parallelization with PyTorch Distributed Data Parallel (DDP) while maintaining compatibility with Stable-Baselines3 (SB3). The framework enables scalable rollout generation and synchronized policy updates across multiple compute nodes without modifying the underlying algorithm. Using 2D3PO, we examine globally normalized advantage estimation, where advantage statistics are aggregated across workers prior to normalization to ensure consistent gradient scaling. The approach is evaluated on a high performance computing cluster across classical control, continuous control, and structured optimization environments. Results show that increasing environment parallelism reduces training runtime by 3x while preserving reward performance relative to a single-node baseline, demonstrating an effective and practical method for accelerating RL training.**

***Index Terms*—component, formatting, style, styling, insert.**

I. INTRODUCTION

Reinforcement learning (RL) has achieved strong performance across domains including control, robotics, and large-scale sequential decision making [1]–[3]. Among policy gradient methods, *Proximal Policy Optimization (PPO)* is widely adopted due to its stability and ease of implementation [4]–[6]. However, PPO training typically requires millions of environment interactions and repeated policy updates, resulting in substantial wall-clock training times that limit experimental throughput. This creates a fundamental bottleneck that restricts large-scale experimentation, slows hyperparameter tuning, and underutilizes available parallel resources in high-performance computing environments [7], [8]. PPO is selected as the focus of this work due to its widespread adoption as a stable on-policy baseline and its representative computational profile: while effective, it is inherently sample- and compute intensive. Improving the efficiency and scalability of PPO training therefore provides a meaningful and broadly relevant target for distributed RL [9].

However, existing distributed RL frameworks, while effective at scaling training [10]–[12], often tightly couple algorithmic logic with system-level execution. The coupling introduces inconsistencies in how rollout data is collected, aggregated, and processed across workers, including differences in batching, synchronization, and normalization [13]. Consequently, the resulting training procedure may deviate from its centralized counterpart, making it difficult to isolate the effects of distribution from underlying algorithmic behavior. The lack of equivalence limits controlled evaluation, as observed differences in performance and efficiency may arise from implementation-specific artifacts rather than the distributed setting itself [14], [15]. As a result, there remains a need for approaches that preserve algorithmic consistency while enabling scalable execution, allowing distributed RL methods to be evaluated under well-defined and reproducible conditions [16].

In addition to system-level considerations, distributed PPO introduces an underexplored algorithmic issue. Advantage normalization is commonly used to stabilize policy gradient updates [17]–[19], but in distributed settings it can be applied either locally per worker or globally across workers. These alternatives impose different tradeoffs between statistical consistency and communication overhead, yet their impact on training behavior has received limited empirical study.

This work addresses these challenges by introducing 2D3PO, a modular distributed PPO framework and toolkit that integrates *Ray*-based environment parallelization with *PyTorch Distributed Data Parallel (DDP)* while preserving compatibility with *Stable-Baselines3 (SB3)* [13], [16], [20], a popular and heavily relied on open-source suite of reinforcement learning algorithms. The framework decouples environment execution, gradient synchronization, and policy optimization, enabling distributed training without modifying the underlying PPO implementation. Within this architecture, globally normalized advantage estimation is implemented to enforce consistent gradient scaling across workers.

This work investigates to what extent distributed PPO with global advantage normalization improves training efficiency while preserving learning performance across diverse RL environments. More broadly, it addresses a fundamental bottleneck in RL, where training inefficiency limits effective use of modern high-performance computing resources, by enabling scalable, distributed training within SB3 while preserving algorithmic consistency. This allows RL workloads to scale with available compute, supporting larger experiments, faster iteration, and more comprehensive evaluation. The proposed framework is evaluated on a multi-node cluster across classical control, continuous control, and structured optimization environments. The primary contributions of this work are: (1) a modular distributed PPO framework that maintains SB3 compatibility while enabling scalable training, (2) an implementation and analysis of global advantage normalization in distributed PPO, and (3) an empirical evaluation of

runtime scaling and learning performance across diverse RL workloads.

II. BACKGROUND

Reinforcement learning (RL) trains a parameterized policy $\pi_\theta(a \mid s)$, where θ denotes the policy parameters and a is the action taken in environment state s, to maximize expected cumulative reward through interaction with an environment. Executing the policy produces trajectories (rollouts) of experience that are used to update the policy via policy gradient methods [17].

Proximal Policy Optimization (PPO) is a widely adopted policy gradient algorithm that stabilizes updates using a clipped surrogate objective, making it a strong and reliable baseline [4], [6], [16]. PPO operates by collecting batches of rollout data under the current policy and performing multiple gradient updates over this batch. To ensure stable learning, updates are constrained relative to the policy that generated the data, allowing efficient optimization while avoiding large policy shifts [4].

Despite its effectiveness, PPO is computationally expensive, requiring large volumes of environment interaction and repeated optimization over collected batches. This leads to substantial training times that limit experimental throughput and motivate the need for more efficient training approaches.

A. Distributed RL Frameworks, Modularity, and Reproducibility

Distributed learning has been widely adopted as a mechanism for scaling machine learning workloads. In distributed deep learning, data-parallel training is a standard approach in which each worker maintains a replica of the model, computes gradients on a subset of data, and participates in collective communication to aggregate gradients across processes [7], [8]. Large-scale systems such as DistBelief demonstrated that distributed stochastic gradient descent can scale to large models and datasets while maintaining effective training performance [9], and subsequent analyses have characterized the scaling behavior of data-parallel training [7]. Modern frameworks such as DDP implement these principles using synchronized gradient aggregation (e.g., all-reduce), enabling updates that are mathematically equivalent to training on a single aggregated batch [8], [21].

Distributed RL extends these ideas by incorporating environment interaction into the training loop. Early approaches such as A3C demonstrated that multiple actors could generate trajectories concurrently to improve throughput [10], while architectures such as IMPALA introduced scalable actor-learner designs for large distributed systems [11]. More recent frameworks, such as RLlib, provide abstractions for distributed rollout collection, scheduling, and cluster-level coordination [12]. Recent work continues to emphasize training efficiency (i.e., reduced wall-clock time) and scalability (i.e., performance as the number of workers or nodes increases), including approaches based on serverless execution and asynchronous coordination strategies for distributed RL [22], [23].

At the same time, reproducibility and experimental rigor remain persistent concerns in RL. Empirical studies have shown that RL results can be highly sensitive to hyperparameters, evaluation protocols, and implementation details, making standardized and interpretable workflows especially important [24]. However, existing distributed RL systems often tightly couple algorithm implementations with system infrastructure, making it difficult to isolate the effects of individual design choices [14]. In contrast, widely used research libraries, such as SB3, provide modular and reproducible implementations of RL algorithms [16], but are designed for single-process execution and require external coordination to support distributed training; this creates a gap between scalable distributed execution and reproducible, modular RL workflows.

The present work addresses this gap by introducing a distributed training framework that preserves the integrity of a widely used PPO implementation while enabling scalable execution. nvironment interaction is handled through Ray, a widely used open-source distributed computing framework for scalable Python applications [13], gradient synchronization is performed using DDP, and policy optimization remains within SB3. By introducing distribution through external coordination rather than modifying algorithm internals, the framework maintains compatibility while enabling controlled evaluation of distributed training behavior. The design enables scalable multi-node training while preserving algorithmic consistency, supporting faster experimentation and allowing distributed RL methods to be evaluated under controlled and reproducible conditions.

B. Advantage Estimation and Normalization

Policy gradient methods optimize a parameterized policy $\pi_\theta(a \mid s)$ by estimating gradients of the expected return objective. The standard policy gradient estimator ∇ can be written as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) A_t], \tag{1}$$

where A_t denotes the advantage function at timestep t, measuring the relative value of an action compared to a baseline value function [17]. The use of advantage functions reduces variance in policy gradient estimates by subtracting a state-dependent baseline, typically the value function, without introducing bias; detailed analysis is provided in prior work [17], [19].

In practice, advantages are not directly available and must be estimated from sampled trajectories. For PPO, this work adopts *Generalized Advantage Estimation (GAE)* as the sole formulation of A_t. GAE computes advantages as an exponentially weighted sum

$$A_t \equiv \hat{A}_t^{GAE_{(\gamma,\lambda)}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+1} \tag{2}$$

of one-step prediction errors

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \tag{3}$$

where $V(s_t)$ denotes the value function, $\gamma \in [0,1]$ is the discount factor, and $\lambda \in [0,1]$ controls the bias–variance tradeoff

[18]. Smaller values of λ reduce variance at the cost of increased bias, while larger values recover Monte Carlo-style estimates with lower bias but higher variance.

Despite the variance reduction introduced by GAE, advantage estimates can still exhibit substantial variability across samples, particularly in high-dimensional or stochastic environments and when using function approximation [6], [15], [19]. The variability can lead to unstable gradient updates and inefficient learning. To further stabilize optimization, it is common practice to normalize advantages within a batch. Given a batch of estimated advantages $\{A_i\}_{i=1}^N$, normalization is typically performed as

$$\tilde{A}_i = \frac{A_i - \mu_A}{\sigma_A + \varepsilon}, \tag{4}$$

where

$$\mu_A = \frac{1}{N}\sum_{i=1}^{N} A_i,\ \sigma_A^2 = \frac{1}{N}\sum_{i=1}^{N}(A_i - \mu_A)^2 \tag{5}$$

and ε is a small constant for numerical stability. The transformation centers the advantages to zero mean and rescales them to unit variance, which helps control the magnitude of policy gradient updates and improves optimization stability in practice [4], [6].

From an optimization perspective, normalization acts as an adaptive rescaling of the policy gradient estimator. Since gradient updates are proportional to A_t scaling advantages directly affects the effective step size. Centering removes bias introduced by skewed advantage distributions, while variance normalization mitigates instability caused by large gradients. These effects are particularly important in PPO, where multiple optimization epochs are performed over the same batch of data.

However, the effectiveness and interpretation of advantage normalization depend critically on how the underlying batch is defined. In standard single-process settings, normalization is applied over trajectories sampled from a single policy, ensuring consistent scaling across all samples. In more complex training settings, the notion of a batch may differ, raising questions about how normalization should be applied to maintain consistency in gradient scaling.

These considerations motivate a closer examination of advantage normalization in distributed training settings.

C. Distributed Training Assumptions and Global Advantage Normalization

In distributed RL, multiple workers interact with independent environment instances to generate trajectories in parallel. Under synchronized parameters, each worker collects data using the same policy, and the aggregated data across workers can therefore be treated as samples drawn from a common distribution induced by π_θ. When environment instances are independent, this aggregated data approximates a single larger batch, allowing standard data-parallel training assumptions to be applied [7], [8], [21].

Let g_w denote the gradient computed by worker w using its local batch of rollout data, and let W denote the total number of workers. The distributed update is then formed by averaging gradients across workers:

$$g_{global} = \frac{1}{W}\sum_{w=1}^{W} g_w, \tag{6}$$

where g_{global} is the aggregated gradient used to update the shared policy parameters. Under synchronized parameters and disjoint local batches, this update is equivalent to computing a gradient over the union of all worker-collected samples, preserving consistency with centralized PPO training.

However, this equivalence does not automatically extend to preprocessing steps that depend on batch-level statistics. As introduced in Section II-B, advantage normalization uses the mean and variance of the advantage values A_t. In distributed settings, the key question is whether these statistics are computed from each worker's local batch or from the full aggregated batch across all workers.

If normalization is performed locally, worker w computes its own statistics

$$\mu_A^{(w)} = \frac{1}{N_w}\sum_{t\in B_w} A_t,\ (\sigma_A^{(w)})^2 = \frac{1}{N_w}\sum_{t\in B_w}(A_t - \mu_A^{(w)})^2, \tag{7}$$

where B_w denotes the local batch on worker w and N_w is the number of workers in w's local batch. Advantages on that worker are then normalized as

$$\tilde{A}_t^{(w)} = \frac{A_t - \mu_A^{(w)}}{\sigma_A^{(w)} + \varepsilon}. \tag{8}$$

The procedure allows each worker to normalize independently, but the resulting scaling may differ across workers because each local batch may have a different advantage distribution.

In contrast, global advantage normalization computes statistics over the aggregated batch across all workers.

The global statistics are then

$$\mu_A = \frac{1}{N}\sum_{t\in B} A_t,\ \sigma_A^2 = \frac{1}{N}\sum_{t\in B}(A_t - \mu_A)^2, \tag{9}$$

and advantages are normalized using these shared values:

$$\tilde{A}_t = \frac{A_t - \mu_A}{\sigma_A + \varepsilon}. \tag{10}$$

The distinction is therefore not in the normalization formula itself, but in the batch over which μ_A and σ_A are computed. Local normalization uses per-worker statistics $\mu_A^{(w)}$ and $\sigma_A^{(w)}$, whereas global normalization uses statistics computed from the full distributed batch. As a result, global normalization ensures that all workers scale advantages with respect to the same distribution, making the normalized advantages consistent across processes; this introduces a tradeoff.

Local normalization avoids additional cross-worker communication, but may produce inconsistent gradient scaling when local batches differ. Global normalization introduces extra synchronization to compute shared statistics, but aligns the distributed update more closely with the interpretation of PPO as operating over a single aggregated batch. Although distributed RL systems have extensively studied scalability and

throughput, this normalization choice has received comparatively limited focused analysis [11], [12].

This work adopts global advantage normalization within a synchronized distributed PPO framework and evaluates its effect on both training efficiency and learning behavior.

D. Reinforcement Learning for Structured Optimization

RL has increasingly been applied to structured optimization problems, including routing, scheduling, and combinatorial decision-making. Early work demonstrated that neural networks trained with RL can learn heuristics for combinatorial optimization problems such as the traveling salesman and vehicle routing problems [3], [25]. More recent studies extend this approach to large-scale graph optimization and distributed decision problems [26]. Survey work indicates that RL is now applied across a wide range of industrial optimization domains, including production planning and resource allocation [27]. These environments differ substantially from classical and continuous control tasks. They often involve discrete action spaces, delayed rewards, and strong dependencies between sequential decisions, resulting in more complex learning dynamics. Consequently, the effects of distributed rollout generation and batch composition may differ from those observed in standard control benchmarks.

Recent work further demonstrates the relevance of RL in manufacturing and digital twin systems. For example, RL has been applied to semiconductor manufacturing planning using digital twin frameworks, highlighting its potential for real-time decision support in complex industrial systems [28]–[31]. These applications underscore the importance of evaluating RL methods in domains beyond traditional benchmarks. However, most distributed RL studies focus primarily on control environments, leaving open questions regarding how distributed training affects learning behavior in structured optimization settings.

This gap motivates the following research question: to what extent does distributed PPO with global advantage normalization improve training efficiency while preserving learning performance across diverse RL environments? We hypothesize that incorporating global advantage normalization within a synchronized distributed PPO framework will reduce overall training runtime through parallelized data collection and optimization, while maintaining performance comparable to centralized training. This expectation is supported by prior work demonstrating that data-parallel training can preserve optimization behavior under synchronized updates [7], [8], [21], as well as studies showing that RL performance can be sensitive to changes in data distribution and training dynamics [14], [15]. As a result, while efficiency gains are expected to be consistent, the degree to which performance is preserved may vary across environment classes.

III. METHODOLOGY

This work proposes 2D3PO, a distributed PPO framework and toolkit, that parallelizes rollout collection and normalizes aggregated advantage estimates. The work then evaluates the new framework by examining its training efficiency and scalability performance in distributed, high-performance environments. Two objectives are considered: (1) determining whether 2D3PO reduces total training runtime relative to a centralized single-node PPO baseline, and (2) analyzing how runtime and performance scale as computational resources are increased.

To address these objectives, a centralized PPO implementation is compared against 2D3PO, which combines Ray-based environment parallelization, DDP-based gradient synchronization, and global advantage normalization. Training efficiency is measured as total wall-clock runtime, while performance is evaluated using reward outcomes across classical control, continuous control, and structured optimization environments. Scalability is assessed by varying the number of environments and compute nodes.

A. 2D3PO: Distributed RL Architecture

2D3PO is a new distributed RL framework designed to enable scalable PPO training while preserving the behavior of a centralized implementation. The architecture separates environment interaction from policy learning, allowing rollout generation to scale across multiple compute nodes while maintaining synchronized model updates.

Fig. 1 illustrates the overall system architecture and data flow. The system consists of two primary components: distributed environment execution and synchronized policy optimization. Environment interaction is parallelized using Ray, where each worker is implemented as a *Ray actor*, a stateful process that independently executes tasks and maintains its own environment instance [13]. Each actor executes an RL training environment and interacts with the current policy to generate trajectory data. A *Ray head* node is used for coordination, managing task scheduling and communication across actors. In 2D3PO, the head designation is a system-level requirement rather than a distinct computational role. The head node participates in training in the same manner as all other nodes, executing rollouts and policy updates. Its primary distinction is that it serves as the designated process for evaluation (rank 0 evaluation), although any node could be assigned this role without affecting training behavior.

The RL training environments follow the Gymnasium (Gym) interface, which provides standardized benchmarks for RL research [32]. These environments define observation spaces (what the agent observes), action spaces (what decisions it can take), and reward functions (how performance is measured), enabling consistent evaluation across diverse problem domains. Common examples include classical control tasks (e.g., CartPole and Acrobot) and continuous control tasks (e.g., LunarLander, HalfCheetah, and Humanoid). In addition to these standard benchmarks, this work also includes structured optimization environments, which are implemented separately but follow the same Gym interface. Representative examples are shown in Fig. 2, and the structural characteristics of all evaluated environments are summarized in Table I.

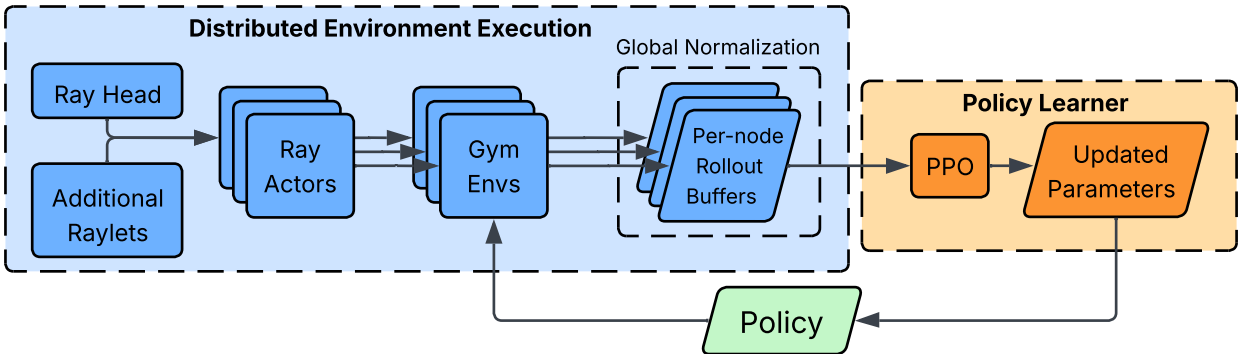
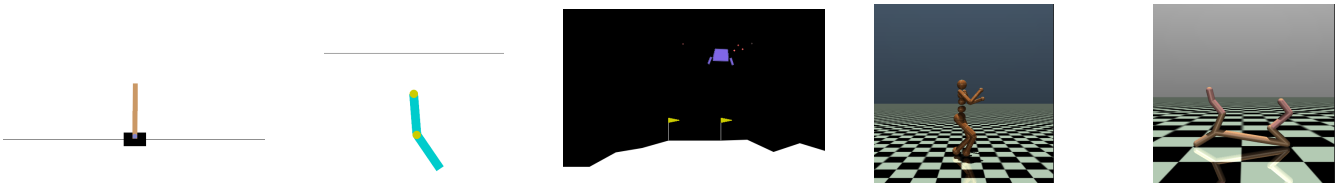


Fig. 1: High-level architecture and data flow of 2D3PO. Ray actors execute environments in parallel across nodes and store transitions in per-node rollout buffers. PPO optimization is performed locally on each node, while model parameters are synchronized across nodes using DDP, enabling distributed rollout collection with globally consistent updates.



(a) CartPole (b) Acrobot (c) Lunar Lander (d) Humanoid (e) Half-Cheetah

Fig. 2: Representative Gymnasium benchmark environments used in the experiments from the classical and continuous control categories. Structured optimization environments are not shown.

Actors are distributed across compute nodes, with each node running a local training process. Transitions generated by actors are stored in node-local rollout buffers, meaning that experience is partitioned across nodes rather than globally shared. Policy optimization is performed locally on each node using the SB3 implementation of PPO [16]. Model parameters are replicated across nodes and synchronized using DDP.

During training, each node computes gradients using its local rollout buffer. These gradients are aggregated across nodes using all-reduce operations, a collective communication primitive that aggregates values across workers and redistributes the result to all processes. This ensures that all nodes apply identical parameter updates, maintaining consistency with a centralized PPO update while enabling distributed data collection. All nodes execute the same learning procedure and contribute equally to gradient computation; no parameter server or centralized learner is used.

Fig. 1 illustrates the overall system architecture and data flow.

B. Training Workflow

Training in 2D3PO follows an iterative cycle consisting of a rollout phase and a policy update phase. Fig. 1 shows the overall workflow, while Fig. 3 highlights the global advantage normalization step within that workflow. During the rollout phase, each Ray actor interacts with its assigned RL training environment using the current policy to generate trajectories of observations, actions, rewards, and value estimates. These transitions are stored in node-local rollout buffers, resulting in distributed data collection across all nodes.

Following rollout collection, the process enters the policy update phase, where the collected trajectories are used to compute advantages and update the policy. Advantage values A_t are computed locally on each node using GAE using equation (2). Since rollout buffers are distributed, advantage values are initially computed with respect to local data only.

To ensure consistency across nodes, advantage normalization is performed globally prior to policy optimization. Rather than normalizing advantages independently on each node, 2D3PO computes normalization statistics across the aggregated batch of all workers. This process is illustrated at a high level in Fig. 1, with Fig. 3 providing an expanded view of the global advantage normalization substep, and is summarized in Algorithm 1. The global normalization procedure proceeds as follows:

1) **Local statistic computation:** Each node computes summary statistics of its locally computed advantages, including the sum, squared sum, and count.
2) **Distributed aggregation:** These statistics are aggregated across all nodes using all_reduce operations, producing global sums and counts.
3) **Global statistic computation:** The aggregated values are used to compute the global mean μ_A and variance σ_A^2 of the advantage distribution.
4) **Normalization:** Advantages within each node's rollout buffer are normalized using the global statistics according to the formulation introduced in Section II-B.

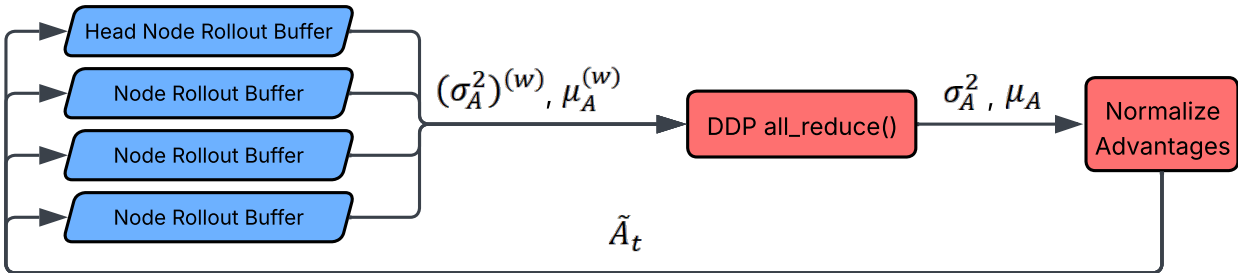


Fig. 3: Expanded view of the global advantage normalization step shown within the 2D3PO workflow in Fig. 1. Each node computes local advantage statistics from its rollout buffer, these statistics are aggregated across nodes using DDP all-reduce, and the resulting global statistics are used to normalize advantages consistently before PPO optimization.

This procedure ensures that all nodes normalize advantages with respect to a consistent distribution, aligning the distributed training process with the behavior of a centralized PPO update.

After normalization, each node performs the policy update using its local rollout buffer and the SB3 implementation of PPO [4], [16]. Gradients are computed independently on each node and then synchronized using DDP. Specifically, an all-reduce operation aggregates gradients across all nodes, after which each node applies the same averaged gradient update to its local model parameters. This guarantees that all model replicas remain identical after each update step, as summarized in Algorithm 1.

Algorithm 1 2D3PO Training Procedure

Require: Distributed nodes W, rollout length, total timesteps

Initialize PPO model on all nodes and synchronize parameters
while training not complete **do**
  Rollout collection (parallel across nodes)
  Collect trajectories into local rollout buffers
  Local advantage computation
  Compute advantages A_t using GAE on each node
  Compute local statistics $(s_1 = \sum A_t,\ s_2 = \sum A_t^2,\ n)$
  Global advantage normalization
  All-reduce (s_1, s_2, n) across nodes
  Compute $\mu_A = s_1/n$, $\sigma_A^2 = s_2/n - \mu_A^2$
  Normalize advantages:
  $\tilde{A}_t = (A_t - \mu_A)/(\sigma_A + \varepsilon)$
  Synchronized policy update
  Compute local PPO gradients
  All-reduce gradients via DDP
  Apply identical parameter updates on all nodes
  Evaluation
  Perform evaluation on rank 0; other nodes wait
end while

The updated policy parameters are then used in the next rollout phase, completing the training cycle. At scheduled evaluation points, model evaluation is performed on the designated rank 0 process, while all other nodes remain synchronized before proceeding. This iterative process enables distributed data collection while maintaining consistent policy updates across all nodes, preserving the learning dynamics of centralized PPO.

C. Implementation Details and Design Choices

Several design choices were made to balance scalability, consistency, and reproducibility. First, a one-to-one mapping between environments and Ray actors ensures uniform workload distribution and simplifies parallelization, as each actor independently manages a single environment instance without requiring dynamic scheduling or load balancing. This mapping enables predictable scaling, where increasing the number of environments directly increases parallel workload. Second, synchronous gradient aggregation via DDP is used to preserve centralized-equivalent training behavior, avoiding issues associated with asynchronous updates such as gradient staleness. In asynchronous settings, gradient staleness refers to the use of gradients computed from outdated model parameters, where some workers update the model using information that no longer reflects the current policy. Finally, global advantage normalization is used to maintain consistent gradient scaling across workers, aligning the distributed update with the interpretation of PPO as operating on a single aggregated batch.

The experimental evaluation uses environments following the Gym/Gymnasium interface to ensure consistent interaction and comparability across tasks. As summarized in Table I, the selected environments are grouped into three categories: classical control (CartPole and Acrobot), continuous control (LunarLander, HalfCheetah, and Humanoid), and structured optimization (LOTZ, FabSim, and CVRP).

Classical control tasks involve low-dimensional states and discrete actions, while continuous control tasks operate over higher-dimensional states with continuous actions. The structured optimization environments are custom implementations that follow the same interface but introduce combinatorial, domain-specific decision-making. For example, FabSim is a variation of the Job Shop Scheduling Problem, where the goal is to optimize a schedule of jobs assigned to machines [28], [30], and LOTZ is a binary optimization problem, where the agent must identify bits to flip to maximize a Boolean logic function. These categories enable evaluation of how distributed training behavior varies across different problem classes.

TABLE I: Structural characteristics of the environments used in the experimental evaluation.

Environment	Category	State Space	Action Space
CartPole	Classical Ctrl.	4D continuous state	Discrete left/right force
Acrobot	Classical Ctrl.	Joint angles and velocities	Discrete torques
LunarLander	Continuous Ctrl.	Position, velocity, angle, contacts	Discrete thrusters
HalfCheetah	Continuous Ctrl.	Joint positions and velocities	Continuous joint torques
Humanoid	Continuous Ctrl.	Full-body joint state	Continuous multi-joint torques
LOTZ	Structured Opt.	Binary bit vector	Discrete bit flips
CVRP	Structured Opt.	Location, capacity, customer demand	Discrete next-customer choice
FabSim	Structured Opt.	Machine, job, and constraint state	Discrete job assignments

D. Hardware Environment and Configurations

Experiments were conducted on a multi-node (num nodes) high-performance computing cluster using the SLURM job manager using Ray v2.54.1 and SB3 v2.7.0. Each node is equipped with an Intel(R) Xeon(R) Gold 5418Y processor, 400 GB of RAM, and up to 48 CPU cores, with nodes interconnected via 100 Gbps HDR InfiniBand. Although each node supports 48 cores, at most 32 CPU cores were utilized per node to maintain consistent workload division across configurations. Each experiment was allocated exclusive access to its assigned nodes to eliminate interference from other jobs.

Four node configurations were evaluated, with one Ray actor per Gym/Gymnasium environment instance:

- 1 node, 16 environments
- 1 node, 32 environments
- 2 nodes, 64 environments
- 4 nodes, 128 environments

Together, these configurations scale both node count and environment count while maintaining a one-to-one mapping between CPU cores and environments.

E. Training and Evaluation Protocol

Each experiment was trained for a fixed budget of 10 million environment timesteps using default SB3 PPO hyperparameters. This training budget was selected as a practical balance between runtime and learning performance. Empirically, 10 million timesteps were sufficient to show representative performance trends across the evaluated environments without introducing excessive wall-clock training time. Multiple independent runs were executed per configuration using different random seeds.

Policy evaluation was performed at scheduled intervals during training. Evaluation intervals were determined by the rollout size $N_{\text{ENVS}} \times N_{\text{STEPS}}$, which typically results in checkpoint timesteps that do not align exactly with uniform intervals. To standardize comparisons, evaluations were reported at target intervals of 1 million timesteps by selecting, for each target, the recorded evaluation closest to that value. This reporting convention is used throughout the reward analyses in Section IV.

At each evaluation point, the policy was executed deterministically for 10 episodes, and the mean $\pm$ standard deviation was recorded. To enable consistent comparison across configurations, reward performance is reported as a ratio relative to the baseline configuration using 16 environments. Specifically, the reward ratio is defined as

$$R_{ratio} = \frac{R_{config}}{R_{baseline}},$$

where values greater than 1 indicate improved performance relative to the baseline. This normalization allows results to be compared across environments with differing reward scales. Statistical comparisons of reward ratios against the 16 environment baseline were conducted using a Dunnett-style test, which is appropriate for multiple comparisons against a single control. Training runtime is measured as the total wall-clock time required to complete the full training process, reported in minutes. It is computed as $T_{total} = T_{end} - T_{start}$.

IV. Results

This section presents results with respect to the two objectives defined in Section III: improving training efficiency through distributed rollout collection and evaluating whether learning performance is preserved as computational resources increase. Section IV-A examines runtime scaling across configurations, and Section IV-A examines reward performance relative to the 16 environment baseline, including statistical comparisons for notable differences.

A. Runtime Scaling

Increasing environment parallelism reduced total wall clock training time across all evaluated environments. The largest gains occurred in the multi-node configurations, with runtimes at 128 environments generally about one third of those at 16 environments.

The largest absolute reductions were observed in higher cost environments. CVRP decreased from 133.34 to 34.66 minutes and FabSim from 133.49 to 35.71 minutes when scaling from 16 to 128 environments. Similar trends were observed in the classical control tasks, with CartPole decreasing from 70.93 to 22.86 minutes and Acrobot from 82.75 to 21.12 minutes. Lunar Lander, HalfCheetah, Humanoid, and LOTZ followed the same overall pattern.

The 32 environment configuration provided only limited gains and was occasionally slightly slower than the 16 environment baseline. In contrast, the 64 and 128 environment configurations consistently produced lower runtimes across environments.

TABLE II: Training runtime (in minutes) across configurations.

Environment	16CPU	32CPU	64CPU	128CPU
Acrobot	82.8 ± 19.1[a]	73.2 ± 3.2	41.5 ± 6.3	21.1 ± 2.8
Cart Pole	70.9 ± 0.4	74.0 ± 0.3	41.6 ± 2.8	22.9 ± 1.6
CVRP	133.3 ± 0.8	132.7 ± 28.9	66.9 ± 12.2	34.7 ± 8.3
FabSim	133.5 ± 5.8	118.4 ± 2.3	64.4 ± 1.0	35.7 ± 0.4
Half-Cheetah	75.9 ± 0.5	79.2 ± 6.4	46.5 ± 2.4	28.0 ± 0.4
Humanoid	107.3 ± 26.8	91.2 ± 3.4	48.4 ± 8.2	29.7 ± 6.3
LOTZ	94.3 ± 0.8	85.8 ± 0.9	47.4 ± 0.6	29.2 ± 0.6
Lunar Lander	80.4 ± 5.6	74.2 ± 0.6	41.4 ± 2.5	23.8 ± 1.8

[a]Table entries are shown as mean ± standard deviation.

Runtime variability differed across environments. CartPole and LOTZ showed low variation across runs, while Acrobot, CVRP, and Humanoid showed greater variability in some lower count configurations. Despite this variability, the overall runtime trend was uniform: increasing distributed parallelism reduced training time in every evaluated environment. Figure 4 and Table II support these observations by showing the full runtime distributions and summary values across configurations.

B. Reward Scaling

Reward performance was generally preserved under distributed scaling, but the effect depended on the environment. In several tasks, reward ratios remained close to 1.0 across training, while a subset of environments showed statistically significant deviations from the 16 environment baseline.

The classical control environments were the most stable overall. CartPole remained effectively unchanged across all configurations, and Acrobot also stayed close to the baseline with only small deviations. Lunar Lander, however, showed a decline in reward as scaling increased, with significant decreases observed at 64 environments ($p = 0.036$) and 128 environments ($p = 0.008$).

The continuous control environments showed greater variability. Humanoid remained near the baseline across configurations, with reward ratios slightly below 1.0 at higher environment counts but without notable statistical differences. HalfCheetah was much less stable, exhibiting high variance across runs and a significant decrease at 128 environments ($p = 0.004$).

The structured optimization environments produced mixed results. CVRP remained close to the baseline across all configurations, indicating that performance was largely preserved. FabSim improved as scaling increased, with significant increases at 64 environments ($p = 0.018$) and 128 environments ($p = 0.012$). LOTZ showed the opposite trend, with reward decreasing as environment count increased and significant reductions at 64 and 128 environments ($p < 0.001$ for both).

Overall, runtime gains were consistent across environments, whereas reward preservation depended on the task. Figure 5 shows the reward trajectories, while Tables III and IV summarize the corresponding reward ratios and reports the notable results from the Dunnett-style comparisons respectively.

TABLE III: Average reward ratio over each checkpoint relative to 16 CPU baseline.

Environment	32 vs. 16 CPU	64 vs. 16 CPU	128 vs. 16 CPU
Acrobot	0.997 ± 0.010[a]	1.040 ± 0.028	1.102 ± 0.065
Cart Pole	0.999 ± 0.001	0.999 ± 0.001	0.999 ± 0.001
CVRP	1.009 ± 0.036	1.014 ± 0.033	1.024 ± 0.019
FabSim	1.033 ± 0.010	1.060 ± 0.018	1.107 ± 0.021
Half Cheetah	1.380 ± 0.441	0.907 ± 0.096	0.838 ± 0.460
Humanoid	1.017 ± 0.030	0.986 ± 0.043	0.945 ± 0.047
LOTZ	0.998 ± 0.025	0.807 ± 0.021	0.552 ± 0.035
Lunar Lander	0.994 ± 0.006	0.966 ± 0.041	0.88 ± 0.122

[a]Table entries are shown as mean ± standard deviation.

TABLE IV: Dunnett-style comparisons against the 16 CPU baseline.

Environment	Comparison	p-value	Direction
HalfCheetah	128 vs. 16 CPU	0.004	Decrease
HalfCheetah	32 vs. 16 CPU	0.089	Decrease
FabSim	64 vs. 16 CPU	0.018	Increase
FabSim	128 vs. 16 CPU	0.012	Increase
Lunar Lander	64 vs. 16 CPU	0.036	Decrease
Lunar Lander	128 vs. 16 CPU	0.008	Decrease
LOTZ	64 vs. 16 CPU	< 0.001	Decrease
LOTZ	128 vs. 16 CPU	< 0.001	Decrease
LOTZ	32 vs. 16 CPU	0.11	Decrease

Only significant and borderline results are shown.

V. Discussion

The results show that distributed PPO with global advantage normalization improves training efficiency across all evaluated environments, but preserves reward performance only in part. With respect to the research objectives, the first objective was clearly achieved, as runtime decreased consistently with distributed scaling. The second objective was only partially achieved, since reward remained near the centralized baseline in some environments but differed significantly in others. The overall answer to the research question is therefore that distributed scaling is effective for reducing wall-clock training time, but the effect of globally normalized advantages and aggregated gradients on reward is environment-dependent.

From a runtime perspective, the outcome is clear. Increasing the number of nodes reduced training time across all environments, with scaling that remained close to linear over much of the evaluated range. Deviations from ideal speedup became more visible at larger scales, primarily because of inter-node communication for gradient synchronization and barrier synchronization during evaluation. Even so, the overall runtime reduction remained substantial, indicating that the distributed framework successfully accelerates PPO training through parallel rollout collection and synchronized updates.

The reward results were more nuanced. Because the rollout batches are constructed to remain equivalent under the distributed formulation, the observed reward differences are more appropriately attributed to how advantage estimates and gradients are aggregated during training than to differences in sampled batch structure. In the distributed setting, advantages are normalized using statistics computed across workers, and gradients are averaged across nodes before each update. The results suggest that these changes do not affect all environ-

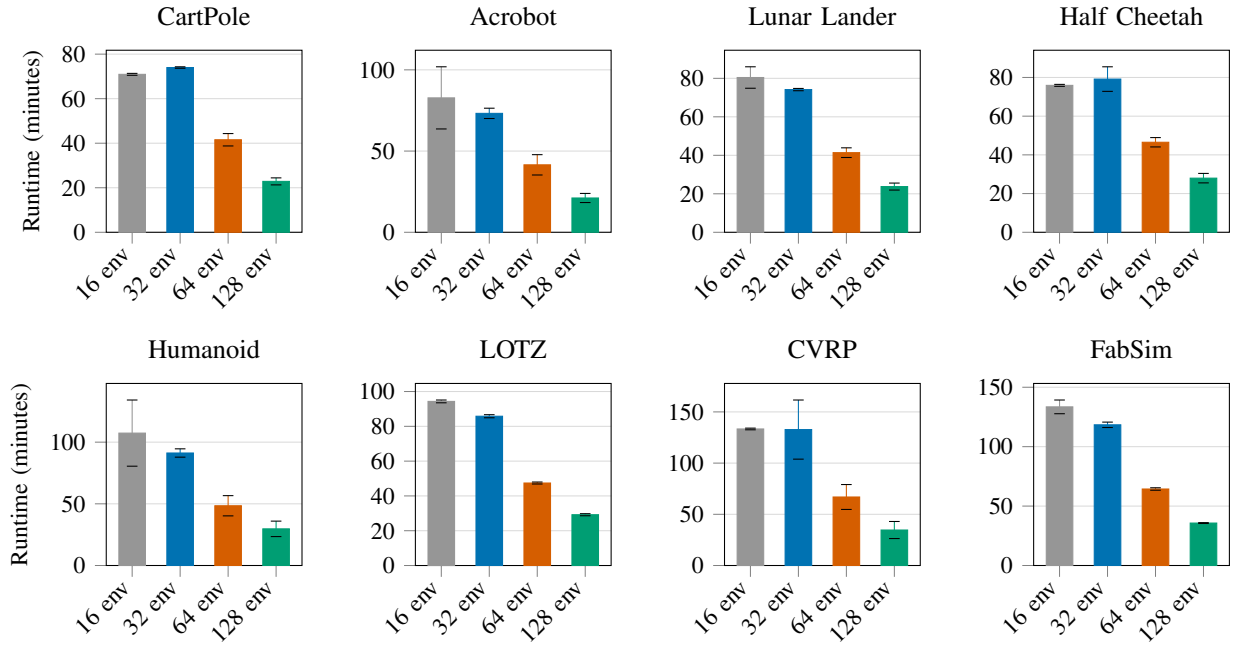


Fig. 4: Final training runtime across distributed configurations for each environment. Error bars indicate ± 1 standard deviation across 10 runs.

ments in the same way.

Within classical control, CartPole and Acrobot remained close to the centralized baseline across all configurations, indicating that globally normalized advantages and synchronized gradient aggregation did not materially alter learning in these tasks. Lunar Lander, however, showed a consistent decrease in reward at higher node counts. This suggests that its optimization behavior is more sensitive to changes in how advantage estimates are scaled and how policy gradients are averaged during distributed updates.

A similar contrast appeared in continuous control. Humanoid remained relatively stable across configurations, whereas HalfCheetah showed greater variability and a significant decrease at the largest scale. In this case, the distributed update process appears to preserve learning performance in one continuous control task but not the other, again indicating that the effect of global advantage normalization and gradient averaging depends on the optimization dynamics of the specific environment.

The structured optimization environments showed the clearest divergence. CVRP remained close to baseline, LOTZ declined steadily as scaling increased, and FabSim improved with additional nodes. Since the distributed setting primarily changes the normalization of advantage estimates and the averaging of gradients, these results suggest that structured optimization tasks can respond very differently to the same aggregation procedure. For LOTZ, the distributed aggregation appears to weaken update effectiveness at larger scales. For FabSim, the opposite trend is observed: the improvement with node count suggests that globally normalized advantages and synchronized gradient updates may produce a more effective optimization signal in this environment. This is particularly notable because it points to a concrete direction for future work, namely identifying what properties of FabSim allow distributed aggregation to improve rather than degrade learning.

The statistical comparisons support these patterns. Significant differences were concentrated in the environments that deviated most clearly from baseline behavior, namely Lunar Lander, HalfCheetah, FabSim, and LOTZ. By contrast, the environments that remained close to baseline, including CartPole, Acrobot, Humanoid, and CVRP, did not show the same level of statistical separation. The statistical results therefore reinforce the broader trend that distributed PPO does not produce a uniform reward response across tasks, even when the underlying rollout formulation is preserved.

Overall, the hypothesis was partially supported. Distributed scaling consistently reduced wall-clock training time, confirming the expected efficiency benefit. However, preservation of reward was not universal. Instead, the results indicate that when PPO is distributed in a way that introduces global advantage normalization and synchronized gradient aggregation, the resulting reward behavior depends on how each environment responds to those aggregated updates.

VI. Conclusion

This work introduces 2D3PO, a distributed RL framework and toolkit that combines parallel environment simulation with data-parallel policy optimization using the distributed training framework Ray and PyTorch Distributed Data Parallel, while preserving compatibility with Stable-Baselines3. The new framework enables PPO to scale across multiple nodes without modifying the underlying algorithm, providing a practical and

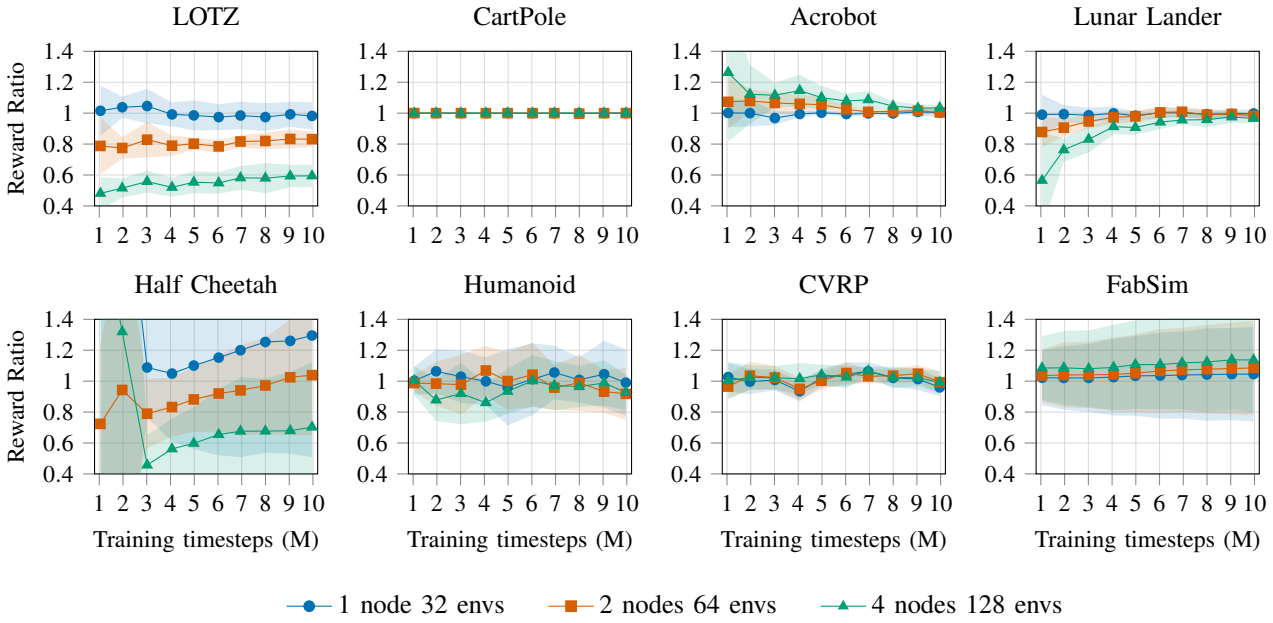


Fig. 5: Reward ratio over training across eight environments. Points use the evaluation nearest each 1M checkpoint; shaded bands show ± 1 standard deviation across 10 runs.

reproducible approach to distributed RL and enabling existing RL codebases to be extended to distributed, high-performance computing environments for training.

The experimental results demonstrate that distributed rollout collection and synchronized updates can reduce training runtime by 3x, with near-linear scaling achieved as we scale from 1 to 4 nodes. At the same time, reward performance is largely preserved across a diverse set of environments, confirming that the proposed approach maintains the learning behavior of PPO under distributed execution in many settings.

The results further show that the effects of distributed training are not uniform across environments. While classical control tasks remain stable, and some structured optimization tasks benefit from increased parallelism, other environments exhibit measurable degradation or increased variability. These findings establish that distributed PPO introduces environment-dependent changes in learning dynamics, even when theoretical assumptions of equivalence are satisfied. The inclusion of multiple environment classes and statistical validation provides a clear and systematic view of how these effects manifest in practice.

Future work will investigate the mechanisms underlying these different environment characteristics in greater detail. This work includes analyzing how rollout structure and trajectory length interact with node-level parallelism, exploring alternative normalization strategies, and extending the evaluation to a broader set of structured optimization problems. Additional studies will also examine whether the new distributed framework on similar environment-dependent effects arise in other RL algorithms and distributed training paradigms.

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015. [Online]. Available: 10.1038/nature14236

[2] M. C. Machado, M. G. Bellemare, E. Talvitie, J. Veness, M. Hausknecht, and M. Bowling, "Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents," 2017. [Online]. Available: https://doi.org/10.48550/arXiv.1709.06009

[3] M. Nazari, A. Oroojlooy, L. Snyder, and M. Takac, "Reinforcement learning for solving the vehicle routing problem," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1802.04240

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: https://doi.org/10.48550/arXiv.1707.06347

[5] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry, "Implementation matters in deep rl: A case study on ppo and trpo," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2005.12729

[6] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem, "What matters for on-policy deep actor-critic methods? a large-scale study," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2006.05990

[7] C. J. Shallue, J. Lee, J. Antognini, J. Sohl-Dickstein, R. Frostig, and G. E. Dahl, "Measuring the effects of data parallelism on neural network training," *Journal of Machine Learning Research*, vol. 20, no. 112, pp. 1–49, 2019. [Online]. Available: https://doi.org/10.48550/arXiv.1811.03600

[8] T. Ben-Nun and T. Hoefler, "Demystifying parallel and distributed deep learning: An in-depth concurrency analysis," *ACM Comput. Surv.*, vol. 52, no. 4, Aug. 2019. [Online]. Available: https://doi.org/10.1145/3320060

[9] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, M. Mao, M. a. Ranzato, A. Senior, P. Tucker, K. Yang, Q. Le, and A. Ng, "Large scale distributed deep networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates,

Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2012/file/6aca97005c68f1206823815f66102863-Paper.pdf

[10] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Harley, T. Lillicrap, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 48. PMLR, 2016, pp. 1928–1937. [Online]. Available: https://proceedings.mlr.press/v48/mniha16.html

[11] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg, and K. Kavukcuoglu, "IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine LearningResearch, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 1407–1416. [Online]. Available: https://doi.org/10.48550/arXiv.1802.01561

[12] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica, "RLlib: Abstractions for distributed reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 3053–3062. [Online]. Available: https://doi.org/10.48550/arXiv.1712.09381

[13] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, "Ray: A distributed framework for emerging AI applications," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, oct 2018, pp. 561–577. [Online]. Available: https://doi.org/10.48550/arXiv.1712.05889

[14] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1709.06560

[15] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 29 304–29 320. [Online]. Available: https://doi.org/10.48550/arXiv.2108.13264

[16] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: https://jmlr.org/papers/v22/20-1364.html

[17] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems*, S. Solla, T. Leen, and K. Müller, Eds., vol. 12. MIT Press, 1999. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf

[18] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1506.02438

[19] E. Greensmith, P. L. Bartlett, and J. Baxter, "Variance reduction techniques for gradient estimates in reinforcement learning," *Journal of Machine Learning Research*, vol. 5, no. Nov, pp. 1471–1530, 2004.

[20] A. Sergeev and M. D. Balso, "Horovod: fast and easy distributed deep learning in tensorflow," 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1802.05799

[21] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "Pytorch distributed: experiences on accelerating data parallel training," *Proc. VLDB Endow.*, vol. 13, no. 12, p. 3005–3018, Aug. 2020. [Online]. Available: https://doi.org/10.14778/3415478.3415530

[22] H. Yu, H. Wang, D. Tiwari, J. Li, and S.-J. Park, "Stellaris: Staleness-aware distributed reinforcement learning with serverless computing," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1–17. [Online]. Available: https://doi.org/10.1109/SC41406.2024.00045

[23] H. Yu, J. Carter, H. Wang, D. Tiwari, J. Li, and S.-J. Park, "Nitro: Boosting distributed reinforcement learning with serverless computing," *Proceedings of the VLDB Endowment*, vol. 18, no. 1, 2024. [Online]. Available: https://doi.org/10.14778/3696435.3696441

[24] A. Patterson, S. Neumann, R. Kumaraswamy, M. White, and A. White, "Cross-environment hyperparameter tuning for reinforcement learning," in *Reinforcement Learning Conference*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2407.18840

[25] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," *CoRR*, vol. abs/1611.09940, 2016. [Online]. Available: https://doi.org/10.48550/arXiv.1611.09940

[26] W. Zheng, D. Wang, and F. Song, "A distributed-gpu deep reinforcement learning system for solving large graph optimization problems," *ACM Transactions on Parallel Computing*, vol. 10, no. 2, 2023. [Online]. Available: https://doi.org/10.1145/3589188

[27] K. T. Chung, C. K. M. Lee, and Y. P. Tsang, "Neural combinatorial optimization with reinforcement learning in industrial engineering: a survey," *Artificial Intelligence Review*, vol. 58, no. 5, p. 130, 2025.

[28] E. Nsiye, T. Wright, B. Tse, and S. Mondesire, "A micro-discrete event simulation environment for production scheduling in manufacturing digital twins," in *Proceedings of MODSIM World*, Norfolk, Virginia, USA, May 20–22 2024.

[29] H.-A. Kuo, T.-Y. Hong, and C.-F. Chien, "A deep reinforcement learning based digital twin framework for resilient production planning under demand uncertainty and an empirical study in semiconductor wafer fabrication," *Computers & Industrial Engineering*, vol. 208, p. 111389, 2025. [Online]. Available: https://doi.org/10.1016/j.cie.2025.111389

[30] S. Mondesire, M. Brown, and B. Soykan, "Fabsim: A micro-discrete event simulator for machine learning dynamic job shop scheduling optimizers," in *Proceedings of the 2025 Winter Simulation Conference (WSC'25)*, Seattle, WA, USA, 2025.

[31] S. Mondesire and B. Soykan, "Automated wafer dispatching through greedy multi-heuristic scheduling in hmlv wafer fabs," in *Proceedings of the Advanced Semiconductor Manufacturing Conference (ASMC): Factory Automation*, Albany, New York, USA, May 11–14 2026.

[32] M. Towers, A. Kwiatkowski, J. U. Balis, G. D. Cola, T. Deleu, M. Goulão, K. Andreas, M. Krimmel, A. KG, R. D. L. Perez-Vicente, J. K. Terry, A. Pierré, S. V. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2407.17032